

Тема 4 - Алгоритмични стратегии и методи в програмирането.

Един много важен аспект от решаването на проблеми е разработването на добри стратегии за проектиране на техния алгоритъм.

Дали алгоритъмът е равен на стратегията?

Алгоритъм: Алгоритъмът е последователност от изчислителни стъпки, които трансформират входа в изход. Това е инструмент за решаване на добре определен изчислителен проблем.

Стратегия: Стратегията е подход (или поредица от подходи), разработен за решаване на изчислителен проблем.

Тема 4 - Алгоритмични стратегии и методи в програмирането.

Алгоритъмът е стратегия, която винаги гарантира правилния отговор.

С какво се различават алгоритмите от стратегиите?

- * Една стратегия може да доведе до неправилни резултати, но правилният алгоритъм винаги ще доведе до правилни резултати.
- * Стратегиите са измислени, алгоритмите са повече или по-малко тествани и надеждни стандарти.
- * Стратегиите са гъвкави, но алгоритмите са фиксирани, т.е. следват само един набор от процедури.

Тема 4 - Алгоритмични стратегии и методи в програмирането.

Например, тривиалният случай на изваждане на две числа, където е включена поне една заемка. Алгоритъм може да бъде традиционният алгоритъм за изваждане:

$$\begin{array}{r} 6 \overset{6}{\cancel{7}} \overset{10+}{1} \\ - 5 \ 2 \ 7 \\ \hline 1 \ 4 \ 4 \\ \hline \end{array}$$

Тема 4 - Алгоритмични стратегии и методи в програмирането.

Въпреки това, можем да измислим други стратегии, примерно:

Place Values

$$\begin{array}{r} 600 - 500 = 100 \\ 70 - 20 = + 50 \\ 1 - 7 = - 6 \\ \hline 144 \end{array}$$

Compensation

$$\begin{array}{r} 527 + 3 = 530 \\ 671 - 530 = 141 \\ + 3 \\ \hline 144 \end{array}$$

Забележете, че при решението с помощта на традиционния алгоритъм за изваждане са необходими малко умствени усилия. Вероятно използваме тази стратегия от началното училище за изваждания. За другите две стратегии се налага да помислим.

Тема 4 - Алгоритмични стратегии и методи в програмирането.

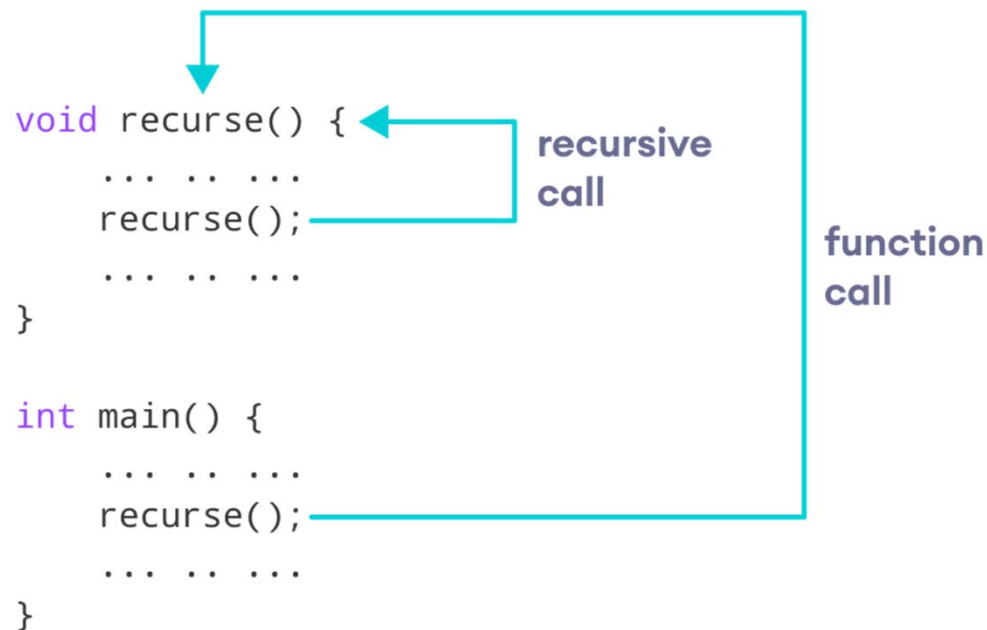
Основни стратегии за изпълнение на изчислителни задачи:

- * Повторение (цикъл)
- * Рекурсия
- * “груба сила” (Brute force)
- * “разделяй и владей” (Divide and Conquer)
- * “обратно проследяване” (Backtracking)
- * „търсене с връщане“ (Backjumping)
- * “обратно верижно свързване” (Backward chaining)
- * “алгоритми на пълно изброяване” (Enumeration algorithm)

Тема 4 - Алгоритмични стратегии и методи в програмирането.

Рекурсия

Функция, която извиква сама себе си, е известна като рекурсивна функция. И тази техника е известна като рекурсия.



Тема 4 - Алгоритмични стратегии и методи в програмирането.

Рекурсията продължава, докато не бъде изпълнено някакво условие.

За да се предотврати безкрайна рекурсия, операторът **if...else** (или подобен подход) може да се използва, когато единият клон прави рекурсивното извикване, а другият не.

```
int main() {  
    ... ..  
    result = factorial(n);  
    ... ..  
}
```

n = 4

**4 * 6 = 24
is returned**

```
int factorial(int n) {  
    if (n > 1)  
        return n * factorial(n-1);  
    else  
        return 1;  
}
```

n = 3

**3 * 2 = 6
is returned**

```
int factorial(int n) {  
    if (n > 1)  
        return n * factorial(n-1);  
    else  
        return 1;  
}
```

n = 2

**2 * 1 = 2
is returned**

```
int factorial(int n) {  
    if (n > 1)  
        return n * factorial(n-1);  
    else  
        return 1;  
}
```

n = 1

**1 is
returned**

```
int factorial(int n) {  
    if (n > 1)  
        return n * factorial(n-1);  
    else  
        return 1;  
}
```


Тема 4 - Алгоритмични стратегии и методи в програмирането.

Предимства на C++ рекурсията:

- * Рекурсията прави нашия код по-кратък и по-чист.
- * Рекурсията е необходима при проблеми, свързани със структури от данни и сложни алгоритми, като обхождане на граф и дърво.

Недостатъци на C++ рекурсията

- * Заема много място в стека в сравнение с итеративна програма.
- * Използва повече процесорно време.
- * Може да бъде по-трудно да се отстранят грешки в сравнение с еквивалентна итеративна програма.

```

#include <iostream>

using namespace std;

int fibonnaci(int x) {
    if((x==1)|| (x==0)) {
        return(x);
    }else {
        return(fibonnaci(x-1)+fibonnaci(x-2));
    }
}

int main() {
    int x , i=0;
    cout << "Enter the number of terms of series : ";
    cin >> x;
    cout << "\nFibonnaci Series : ";
    while(i < x) {
        cout << " " << fibonnaci(i);
        i++;
    }
    return 0;
}

```

Редица на Фибоначи, използваща рекурсия в C++

Серия от числа на Фибоначи е поредица от числа, така че всяко число е сбор от двете предходни, започвайки от нула (0) и едно (1).

Изход

Въведете броя на членовете на серията: 15

Серия на Фибоначи : 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Тук в горната програма функцията "фибоначи" е рекурсивната функция, която се извиква и намира редицата на Фибоначи.

Времовата сложност по рекурсивната програма на Фибоначи е $O(n^2)$ или експоненциална.

```
#include <iostream>
```

```
using namespace std;
```

```
int fact(int n);
```

```
int main()
```

```
{
```

```
    int n;
```

```
    cout << "Enter a positive integer: ";
```

```
    cin >> n;
```

```
    cout << "Factorial of " << n << " = " << fact(n);
```

```
    return 0;
```

```
}
```

```
int fact(int n)
```

```
{
```

```
    if(n > 1)
```

```
        return n * fact(n - 1);
```

```
    else
```

```
        return 1;
```

```
}
```

Програма за изчисление на факториал, използваща рекурсия в C++

Факториал е произведението на цяло число и всички други цели числа под него. Например факториелът на 5 (5!) е равен на $5 \times 4 \times 3 \times 2 \times 1$, т.е. 120.

Изход

Въведете положително цяло число: 6

Факториел от 6 = 720

Тук рекурсивната функция "fact" ще приеме число като параметър, за който искаме да намерим факториела. И след това рекурсивно извикаме функцията, докато основното условие стане вярно, което е, когато числото като параметър стане 1.

Времовата сложност на програмата е $O(n)$, тъй като в даден момент се изпълнява само един цикъл, докато се намира факториелът на числото с помощта на рекурсивната функция. Освен това не се изисква допълнително пространство по време на рекурсивни повиквания и следователно сложността на пространството също е $O(n)$.

```

#include <iostream>

using namespace std;

int calculate(int, int);

int main()
{
    int base, power, result;

    cout << "Enter base number: ";
    cin >> base;

    cout << "Enter power number(positive integer): ";
    cin >> power;

    result = calculate(base, power);

    cout << base << "^" << power << " = " << result;
    return 0;
}

int calculate(int base, int power)
{
    if (power != 0)
        return (base*calculate(base, power-1));
    else
        return 1;
}

```

В тази програма ще изчислим степента на числото, като използваме подхода на рекурсия, при който основата и степента се приемат като входни данни от потребителя.

Изход

Въведете положително цяло число: 3

Въведете степенно число (цяло положително число): 4

$3^4 = 81$

Тук рекурсивната функция "calculate" се извиква отново и отново с основното условие за проверка на степента до 0, тъй като степента винаги е положително цяло число.

Сложността на времето за изпълнение на програмата за намиране на степента на числото с помощта на рекурсия е $O(\log n)$, тъй като при всяко извикване на рекурсивната функция параметърът на следващото извикване се увеличава експоненциално. Следователно времевата сложност е функция на логаритъм.

```

#include <iostream.h>

using namespace std;

int reverseNumber(int n) {
    static temp,sum;

    if(n>0){
        temp = n%10;
        sum = sum*10 + temp;
        reverseNumber(n/10);
    } else {
        return sum;
    }
}

int main() {
    int n,reverse;

    cout<<"Enter number";
    cin >> n;
    reverse = reverseNumber(n);
    cout << "Reverse of number is" << reverse;
    return 0;
}

```

В тази програма ние ще вземем вход от потребителя като цяло число и ще го обърнем с помощта на рекурсивната функция.

Изход

Въведете цяло число: 3456

Обърнатото число е: 6543

В тази програма ще извикаме рекурсивно функцията "reverseNumber" с параметъра, въведен от потребителя.

Сложността на времето за изпълнение на програмата е $O(\log(n))$, защото всеки път, когато функцията се извиква рекурсивно, тя взема $1/10$ от числото като параметър за следващото извикване. Следователно времевата сложност за обръщане на числото с помощта на рекурсивната функция е $O(\log(n))$.

```

#include <bits/stdc++.h>

using namespace std;

bool isprime(int n, int i = 2)
{
    if (n <= 2)
        return (n == 2) ? true : false;

    if (n % i == 0)
        return false;

    if (i * i > n)
        return true;

    return isprime(n, i + 1);
}

int main()
{
    int n = 18;
    if (isprime(n))
        cout << "Yes, Number is Prime Number";
    else
        cout << "No, Number is not Prime Number";
    return 0;
}

```

Проверка дали числото е просто или не с помощта на рекурсия в C++

Просто число е число, което се дели само на себе си и на 1. В тази програма ще проверим дали даденото число е просто число или не.

Тук създаваме рекурсивната функция "isprime" и след това проверяваме условието за просто число с помощта на условни изрази.

Сложността на времето за изпълнение на програмата за проверка дали числото е просто или не е $O(\sqrt{n})$, защото когато рекурсивно извикваме функцията isPrime, ние проверяваме дали е по-малко от корен квадратен от даденото число.

Тема 4 - Алгоритмични стратегии и методи в програмирането.

Заключение

Въпреки че подходът на рекурсия консумира много място за съхранение на паметта поради използването на стекова структура от данни, той се използва широко за решаване на различни проблеми, за да се намали сложността на времето и редовете от кодове в програмата.

Тема 5 - Стратегии в програмирането

Стратегия “груба сила” (*Brute force*)

Описва примитивен стил на програмиране, при който програмистът разчита на изчислителната мощност на компютъра, вместо да използва собствения си интелект, за да опрости проблема, често пренебрегвайки проблеми с мащаба и прилагайки наивни методи, подходящи за малки проблеми, директно към големи.

Тема 5 - Стратегии в програмирането

Стратегия “груба сила” (*Brute force*)

Класически пример в компютърните науки е проблемът на пътуващия търговец (TSP). Да предположим, че един продавач трябва да посети 10 града в страната. Как се определя редът, в който тези градове трябва да бъдат посетени, така че общото изминато разстояние да бъде сведено до минимум?

Решението за груба сила е просто да се изчисли общото разстояние за всеки възможен маршрут и след това да се избере най-краткият. Това не е особено ефективно, защото е възможно да се премахнат много възможни маршрути чрез хитри алгоритми.

Времовата сложност на грубата сила е $O(n*m)$. Така че, ако търсим низ от "n" знака в низ от "m" знака, използвайки груба сила, това ще ни отнеме $n * m$ опита.

Тема 5 - Стратегии в програмирането

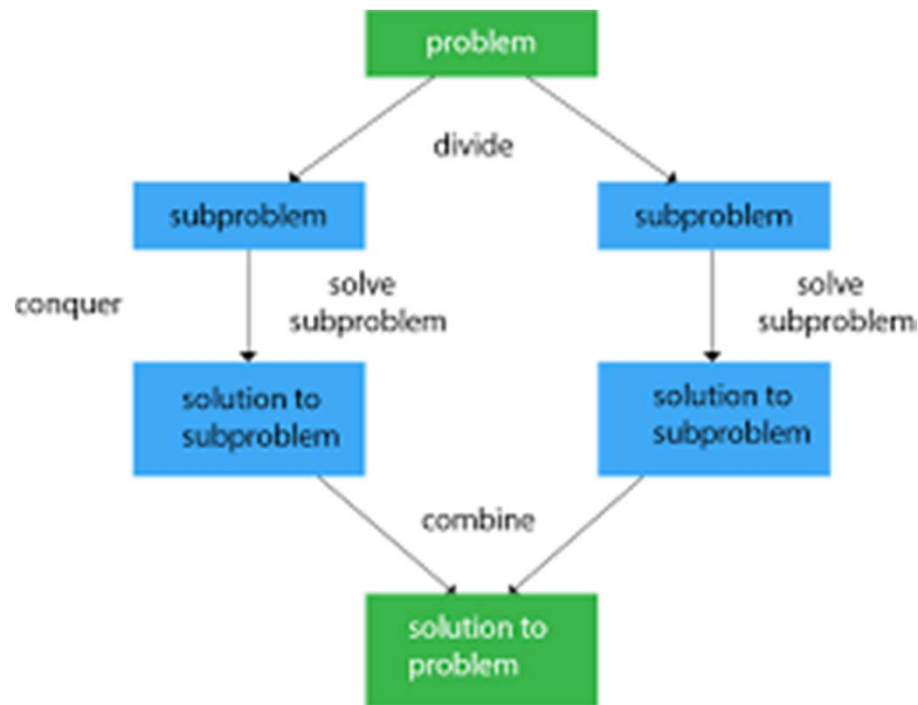
Стратегия “груба сила” (*Brute force*)

Например, представете си, че имате малък катинар с 4 цифри, всяка от 0-9, като се забравили. Тъй като не можете да си спомните нито една от цифрите, трябва да използвате метода на груба сила, за да отворите ключалката. Така че задавате всички числа обратно на 0 и ги опитвате едно по едно: 0001, 0002, 0003 и така нататък, докато се отвори. В най-лошия случай ще са необходими 10^4 или 10 000 опита, за да намерите вашата комбинация.

Тема 5 - Стратегии в програмирането

Стратегия “разделяй и владей” (Divide and Conquer)

В компютърните науки разделяй и владей е парадигма за проектиране на алгоритъм. Алгоритъмът "разделяй и владей" рекурсивно разбива проблем на два или повече подпроблеми от същия или свързан тип, докато те станат достатъчно прости, за да бъдат решени директно.



Тема 5 - Стратегии в програмирането

Стратегия “разделяй и владей” (Divide and Conquer)

Парадигмата “разделяй и владей” често се използва за намиране на оптимално решение на проблем. Основната му идея е да разложи даден проблем на два или повече подобни, но по-прости подпроблема, да ги реши последователно и да състави техните решения за решаване на дадения проблем. Проблемите с достатъчна простота се решават директно.

Например, за да сортирате даден списък от n естествени числа, разделете го на два списъка с около $n/2$ числа всеки, сортирайте всеки от тях на свой ред и вмъкнете двата резултата по подходящ начин, за да получите сортираната версия на дадения списък. Този подход е известен като алгоритъм за сортиране чрез сливане.

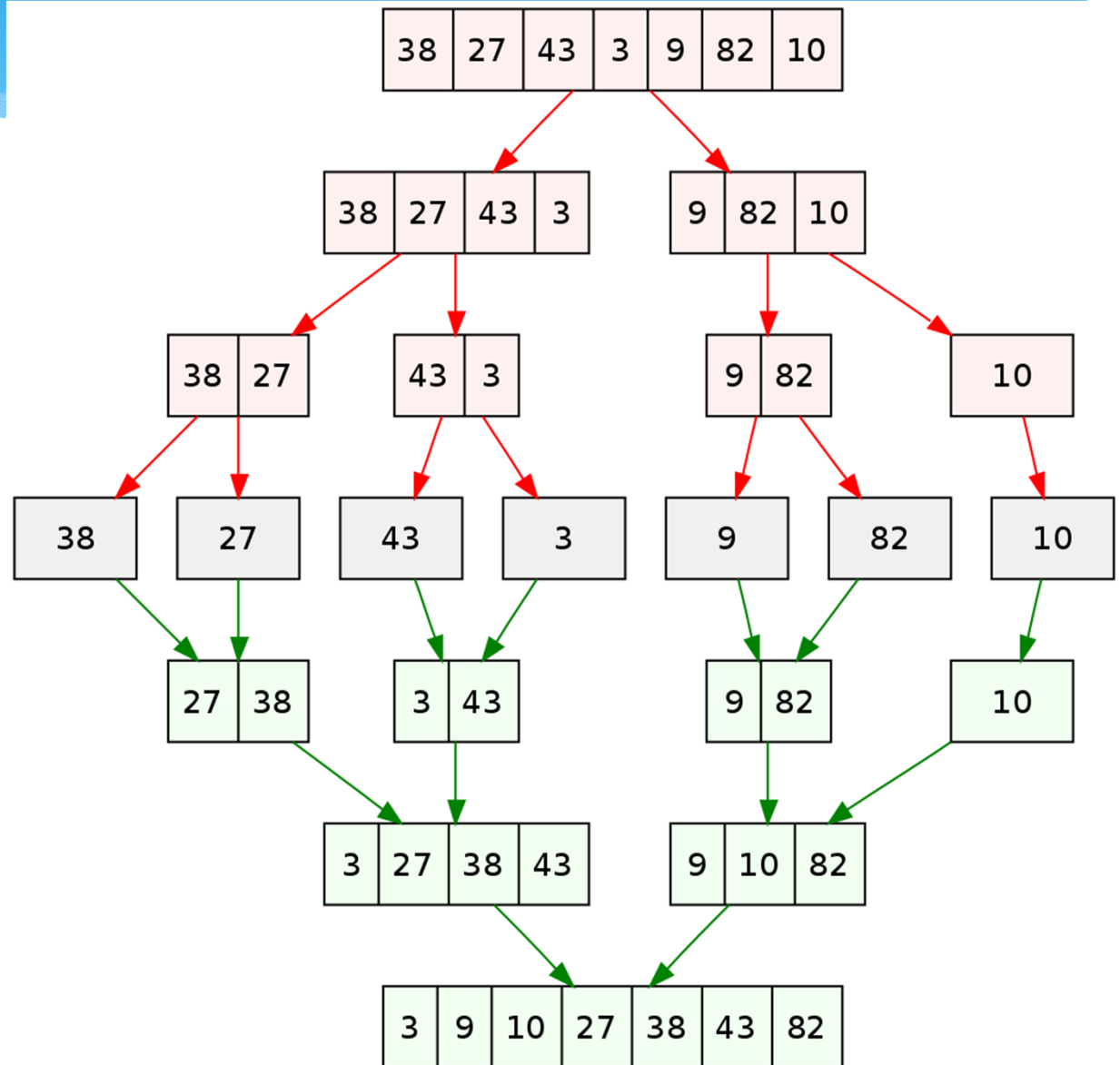
Тема 5 - Стратегии в програмирането

Стратегия

“разделяй и владей”

(Divide and Conquer)

за сортиране на
списъка (38, 27, 43, 3, 9,
82, 10) във възходящ
ред. Горна половина:
разделяне на
подсписъци; mid:
едноелементен списък е
тривиално сортиран;
долна половина:
съставяне на сортирани
подсписъци.



Тема 5 - Стратегии в програмирането

Стратегия “разделяй и владей” (Divide and Conquer)

Решаване на трудни проблеми

Разделяй и владей е мощен инструмент за решаване на концептуално трудни проблеми: всичко, което изисква, е начин за разделяне на проблема на подпроблеми, за решаване на тривиалните случаи и за комбиниране на подпроблеми към първоначалния проблем.

Паралелизъм

Алгоритмите "разделяй и владей" са естествено адаптирани за изпълнение в многопроцесорни машини, особено системи със споделена памет, където комуникацията на данни между процесорите не е необходимо да се планира предварително, тъй като отделни подпроблеми могат да бъдат изпълнени на различни процесори.

Тема 5 - Стратегии в програмирането

Стратегия “разделяй и владей” (Divide and Conquer)

Достъп до паметта

Алгоритмите "разделяй и владей" естествено са склонни да използват ефективно кеш паметта. Причината е, че след като даден подпроблем е достатъчно малък, той и всички негови подпроблеми могат по принцип да бъдат разрешени в кеша, без достъп до побавната основна памет.

Алгоритмите "разделяй и владей" естествено се изпълняват като рекурсивни процедури. В този случай частичните подпроблеми, водещи до решавания в момента, автоматично се съхраняват в стека за извикване на процедури.

Тема 5 - Стратегии в програмирането

Стратегия “разделяй и владей” (Divide and Conquer)

При рекурсивни реализации на D&C алгоритми трябва да се уверите, че има достатъчно памет, разпределена за рекурсивния стек, в противен случай изпълнението може да се провали поради препълване на стека. D&C алгоритмите, които са ефективни във времето, често имат относително малка дълбочина на рекурсия.

Алгоритмите „разделяй и владей“ също могат да бъдат реализирани от нерекурсивна програма, която съхранява частичните подпроблеми в структура от данни, като стек, опашка или приоритетна опашка.

Тема 5 - Стратегии в програмирането

Стратегия “обратно проследяване” (*Backtracking*)

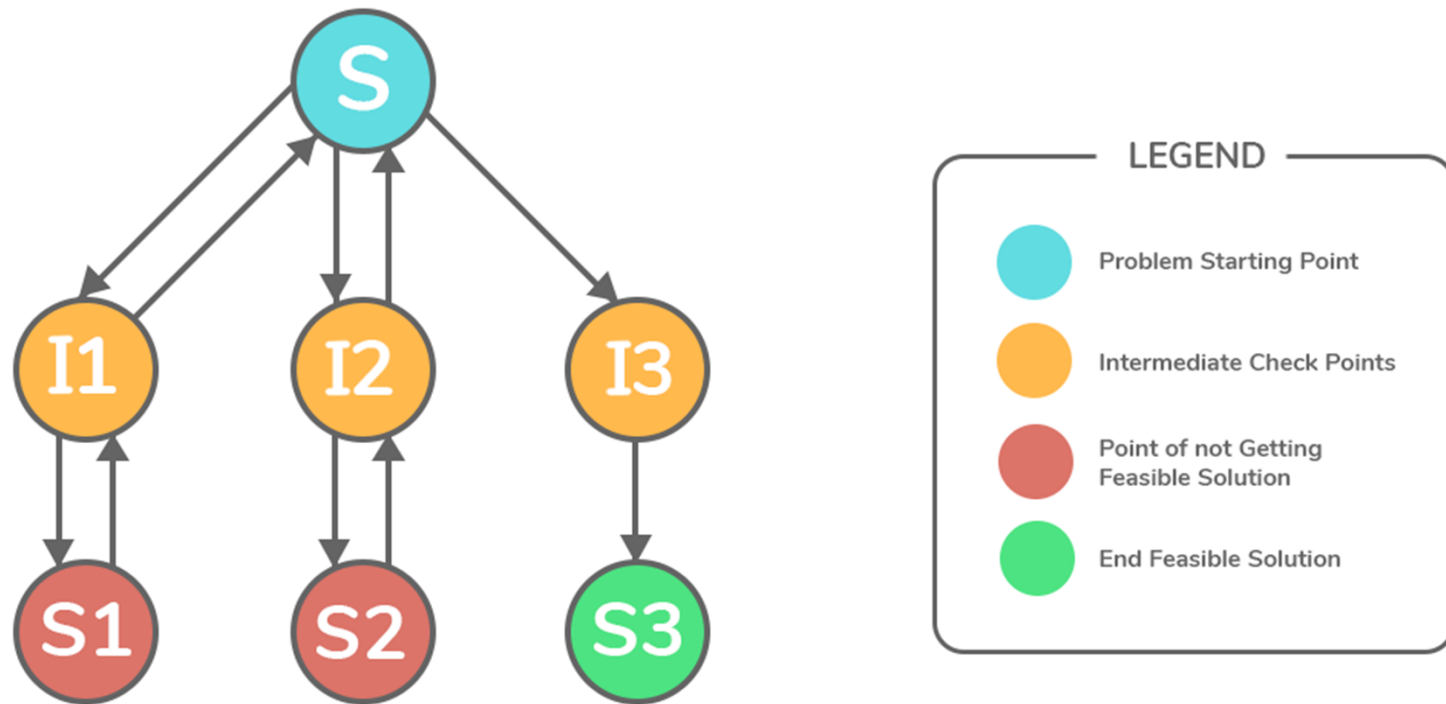
Стратегията за обратно проследяване служи за решаване на проблеми, като използва подход на груба сила за намиране на желания резултат. Подходът на груба сила изпробва всички възможни решения и избира желаните/най-добрите решения.

Използва се рекурсивно извикване, за да се намери крайното решение, като изгражда решение стъпка по стъпка, увеличавайки стойностите с времето. Тук се премахва решенията, които не водят до решение на проблема въз основа на ограниченията, дадени за решаване му.

Тема 5 - Стратегии в програмирането

Стратегия “обратно проследяване” (*Backtracking*)

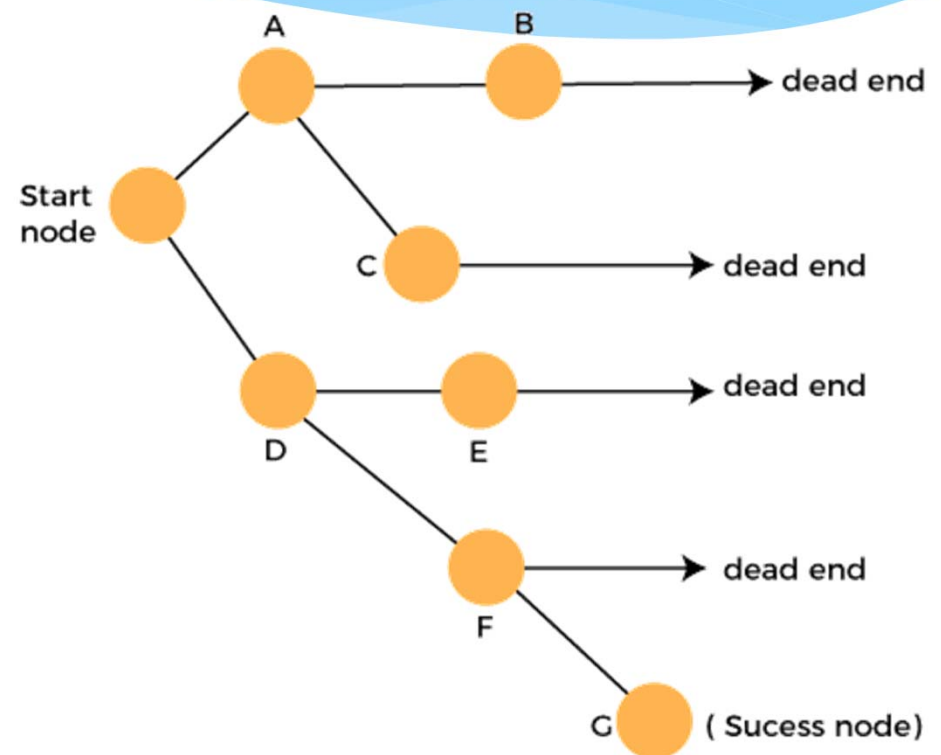
Backtracking



Тема 5 - Стратегии в програмирането

Стратегия “обратно проследяване” (Backtracking)

Самото име за обратно проследяване подсказва, че се връщаме назад и идваме напред; ако удовлетворява условието, тогава връща успех, в противен случай се връщаме отново.



Тема 5 - Стратегии в програмирането

Стратегия “обратно проследяване” (*Backtracking*)

Кога да използвам алгоритъм от тип «обратно проследяване»?

Когато имаме множество възможности за избор, тогава вземаме решенията от наличните възможности за избор. В следните случаи трябва да използваме алгоритъма за обратно проследяване:

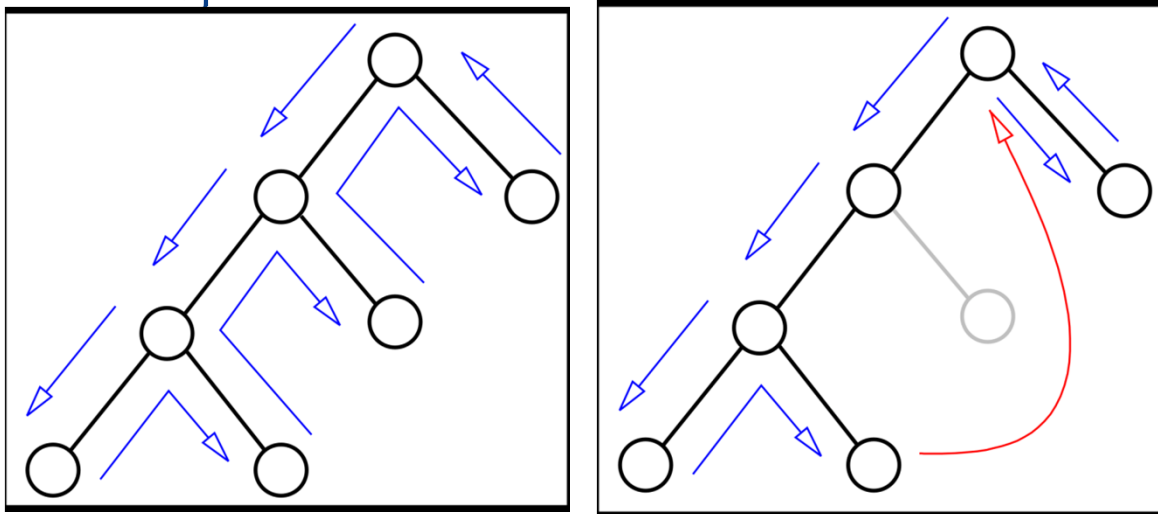
- * Няма достатъчно информация, за да направим най-добрия избор, така че използваме стратегията за обратно проследяване, за да изпробваме всички възможни решения.
- * Всяко решение води до нов набор от избори. След това отново се връщаме назад, за да вземем нови решения. В този случай трябва да използваме стратегията за обратно проследяване.

Тема 5 - Стратегии в програмирането

Стратегия „обратно прескачане“ (*Backjumping*)

В алгоритмите за “обратно проследяване” стратегията „**обратно прескачане**“ е техника, която намалява пространството за търсене, като по този начин повишава ефективността.

Докато в “обратно проследяване” винаги се изкачваме с едно ниво нагоре в дървото на търсене, когато всички стойности за дадена променлива са тествани, при „**обратно прескачане**“ може да се върнем на повече нива нагоре.



Тема 5 - Стратегии в програмирането

Ефективността на алгоритъма за **„обратно прескачане“** зависи от това колко високо е в състояние да скача назад.

Установяването дали даден скок е безопасен не винаги е осъществимо, тъй като безопасните скокове се дефинират по отношение на набора от решения, което алгоритъмът се опитва да намери.

На практика алгоритмите за **„обратно прескачане“** използват най-ниския индекс, който могат ефективно да докажат като безопасен скок. Различните алгоритми използват различни методи за определяне дали скокът е безопасен. Тези методи имат различна цена, но по-високата цена за намиране на по-висок безопасен скок може да бъде заменена с намален обем на търсене поради пропускане на части от дървото на търсене.

Алгоритъмът за **„обратно прескачане“** е подобрение на стратегията за „обратно проследяване“. Той подобрява ефективността, тъй като намалява пространството, използвано от рекурсивния стек за достигане на решение.

Тема 5 - Стратегии в програмирането

Стратегия “обратно верижно свързване” (*Backward chaining*)

Алгоритъмът за “обратно верижно свързване” е форма на стратегическо разсъждение върху задача, което започва с целта и работи в обратна посока, верижно преминавайки през правила, за да намери известни факти, които подкрепят целта.

Свойства на обратното верижно свързване: Известно е като подход отгоре надолу. Обратно верижното свързване се основава на правилото за извод на *modus ponens* (поставяне на способ).

Тема 5 - Стратегии в програмирането

“Обратно верижно свързване” (или обратно разсъждение) е метод за извод, описван разговорно като работа назад от целта. Използва се в автоматизирани системи за доказване на теореми, машини за изводи, асистенти за доказване и други приложения с изкуствен интелект.

В теорията на игрите изследователите я прилагат към (по-прости) под-игри, за да намерят решение на играта, в процес, наречен обратна индукция. В шаха се нарича ретрограден анализ и се използва за генериране на бази за таблици за шахматни ендшпили за компютърен шах.

Тема 5 - Стратегии в програмирането

“Обратно верижно свързване” започва със списък от цели (или хипотеза) и работи в обратна посока от следствието към предходното, за да види дали някакви данни подкрепят някое от тези следствия.

Машина за извод, използваща обратно верижно свързване, ще търси правилата за извод, докато не намери такова с последващо (клауза Then), което съответства на желаната цел. Ако се знае, че предшественикът (If клауза) на това правило е верен, тогава той се добавя към списъка с цели (за да бъде потвърдена нечия цел, трябва също да предостави данни, които потвърждават това ново правило).

Тема 5 - Стратегии в програмирането

Да предположим например, че нов домашен любимец, Фриц, е доставен в непрозрачна кутия заедно с два факта за Фриц:

1. Фриц грачи
2. Фриц яде мухи

Целта е да се реши дали Фриц е зелен въз основа на база от правила, съдържаща следните четири правила:

1. Ако X кряка и X яде мухи – тогава X е жаба
2. Ако X чурулика и X пее – тогава X е канарче
3. Ако X е жаба – тогава X е зелено
4. Ако X е канарче – тогава X е жълто

Тема 5 - Стратегии в програмирането

С обратни разсъждения, машина за изводи може да определи дали Fritz е зелен в четири стъпки. Като начало, запитването е формулирано като твърдение за цел, което трябва да бъде доказано: „Фриц е зелен“.

1. Фриц е заменен с X в правило #3, за да види дали неговото следствие отговаря на целта, така че правило #3 става:

Ако Фриц е жаба – тогава Фриц е зелен

Тъй като следствието съвпада с целта („Фриц е зелен“), механизмът на правилата сега трябва да види дали предшестващото („Фриц е жаба“) може да бъде доказано. Следователно предшественикът се превръща в новата цел:

Фриц е жаба

Тема 5 - Стратегии в програмирането

2. Отново замествайки Fritz с X, правило #1 става:

Ако Фриц кряка и Фриц яде мухи – тогава Фриц е жаба

Тъй като следствието съответства на текущата цел („Фриц е жаба“), машината за изводи сега трябва да види дали предшестващото („Фриц грачи и яде мухи“) може да бъде доказано. Следователно предшественикът се превръща в новата цел:

Фриц грачи, а Фриц яде мухи

Тема 5 - Стратегии в програмирането

3. Тъй като тази цел е комбинация от две твърдения, машината за изводи я разделя на две подцели, като и двете трябва да бъдат доказани:

Фриц грачи

Фриц яде мухи

Тема 5 - Стратегии в програмирането

4. За да докаже и двете от тези подцели, машината за изводи вижда, че и двете от тези подцели са дадени като първоначални факти. Следователно връзката е вярна:

Фриц грачи и Фриц яде мухи

следователно антецедентът на правило #1 е верен и следствието трябва да е вярно:

Фриц е жаба

следователно антецедентът на правило #3 е верен и следствието трябва да е вярно:

Фриц е зелен

Тема 5 - Стратегии в програмирането

Следователно това извеждане позволява на машината за изводи да докаже, че Фриц е зелен. Правила #2 и #4 не са използвани.

Тъй като списъкът с цели определя кои правила се избират и използват, този метод се нарича целево ориентиран, за разлика от управлението от данни извод за верижно предаване. Подходът за **“обратно верижно свързване”** често се използва от експертни системи.

Програмни езици като Prolog, Knowledge Machine и ECLiPSe поддържат обратно верижно свързване в рамките на своите машини за изводи.

Тема 5 - Стратегии в програмирането

Forward Chaining и **Backward Chaining** са двете най-важни стратегии в областта на изкуствения интелект и се използват в областта на експертната система на AI. Те са стратегиите, използвани от машината за изводи при извършване на изводите.

Машина за изводи (Inference Engine) е компонент на експертната система, който прилага логически правила към базата знания, за да изведе нова информация. Той интерпретира и оценява фактите в базата от знания, за да даде отговор.

Базата от знания е структурирана колекция от факти за домейна на системата.

При Forward Chaining механизмът за изводи преминава през всички факти, условия и изводи, преди да изведе резултата, т.е. когато въз основа на наличните данни се вземе решение, тогава процесът се нарича Forwarding chaining, той работи от първоначално състояние и достига до целта (крайния решение).

При Backward Chaining системата за изводи знае крайното решение или цел, тази система започва от целта и работи назад, за да определи какви факти трябва да бъдат утвърдени, за да може целта да бъде постигната, т.е. тя работи от целта (окончателното решение) и достига до първоначалното състояние.

Тема 5 - Стратегии в програмирането

Алгоритми на “пълно изброяване” (Enumeration algorithm)

В компютърните науки алгоритъмът за “пълно изброяване” е алгоритъм, който изброява отговорите на изчислителен проблем. Формално, такъв алгоритъм се прилага за проблеми, които приемат вход и произвеждат списък с решения, подобно на функционалните проблеми. За всеки вход алгоритъмът за изброяване трябва да изведе списък с всички решения без дублиране и след това да спре.

Ефективността на алгоритъм за “пълно изброяване” се измерва:

- * по отношение на времето, необходимо за получаване на решенията, или
- * по отношение на общото време, необходимо за получаване на всички решения, или
- * по отношение на максималното забавяне между две последователни решения и
- * по отношение на времето за предварителна обработка , отчитано като времето преди извеждането на първото решение.

Тази сложност може да бъде изразена по отношение на размера на входа, размера на всеки отделен изход или общия размер на набора от всички изходи, подобно на това, което се прави с чувствителните към изхода алгоритми.